

- 01 Step 0 — 환경 진단 + Stitch 점검
- 02 Step 1 — 화면 분석과 Task 매칭
- 03 Step 2 — GitHub Issue 등록 (화면 라벨)
- 04 Step 3 — Stitch로 첫 화면 시안
- 05 Step 4 — 첫 Task 빌드와 PR 워크플로
- 06 Step 5 — 셀프 코드 리뷰
- 07 Step N — 정리와 비용 비교
- 08 과제 + Tips 5

— STEP 0

환경 진단

자료 4 환경 점검 + 토큰 베이스라인 + Stitch 점검

- 01 0-1. 환경 확인 명령 (node·claude·git·gh)
- 02 0-2. planning_project 폴더 진입 (D-043)
- 03 0-3. 자료 4 산출물 5개 점검
- 04 0-4. 베이스라인과 Stitch 점검

— 활동 0-1 · 산출물: 4 도구 확인 완료

환경 확인 명령 — node·claude·git·gh

통과 조건: node 18.0 이상, claude·git·gh 모두 동작.

MACOS / LINUX

node --version
claude --version
git --version
gh --version

WINDOWS POWERSHELL

node --version
claude --version
git --version
gh --version

활동 0-2 · 산출물: 작업 폴더 진입

planning_project 폴더 진입 (D-043)

자료 4에서 만든 폴더 그대로. 새 폴더 생성 안 함.

MACOS / LINUX

cd planning_project

WINDOWS POWERSHELL

Set-Location planning_project

— 활동 0-3 · 산출물: 자료 4 점검 5개

자료 4 산출물 점검 (D-044 §3.5/§3.6/§3.7)

비어 있는 항목 발견 시 자료 4 해당 Step으로 회귀 후 본 단계 진입.

01	CLAUDE.md (검증 통과)	있는가
02	docs/ (prd, persona, painpoints)	3개 파일 있는가
03	rules/ (3개 파일)	있는가
04	.claude/settings.json	있는가
05	GitHub 저장소 + 초기 커밋	푸시됐는가

— 활동 0-4 · 산출물: 베이스라인 + STITCH 접속

베이스라인과 Stitch 점검

토큰 베이스라인 측정 + Stitch 또는 폴백 도구 접속 확인.

토큰 베이스라인	STITCH 접속
/cost	stitch.withgoogle.com
출력 결과 메모 → Step N에서 비교.	안 되면 v0 또는 Lovable로 폴백.

STEP 0 · 검증 / 성공 기준

Step 1 진입 가능한가

4개 항목 모두 통과 시 Step 1 (화면 분석과 Task 매칭)로 진입.

- 01

☒

planning_project 폴더에서 작업 중인가
- 02

☒

자료 4 산출물 5개가 모두 있는가
- 03

☒

Stitch 또는 폴백 도구에 접속 가능한가
- 04

☒

/cost 베이스라인을 메모했는가

— STEP 1

화면 분석과 Task 매칭

PRD 핵심 기능 3개에서 화면을 도출 → 화면 단위 docs/tasks.md 재작성

- 01 1-1. 화면 분석이 필요한 이유
- 02 1-2. PRD 핵심 기능 재확인
- 03 1-3. 화면 도출 명령
- 04 1-4. 첫 화면 결정 기준
- 05 1-5. Task 화면 단위 재구성
- 06 1-6. 화면-Task 매핑 검토

— 활동 1-1 · 산출물: 분석 필요성 인식

화면 분석이 필요한 이유

기능 단위 ↔ 화면 단위의 매핑이 빌드 직전 필수 절차.

- 1 PRD는 "기능 단위"로 정의
- 2 빌드는 "화면 단위"로 진행
- 3 한 화면에 여러 기능 / 한 기능이 여러 화면
- 4 이 매핑이 명확하지 않으면 매번 흔들림

| 화면 분석은 빌드 전의 필수 절차이다.

활동 1-2 · 산출물: 핵심 기능 3개 한 줄 요약

PRD 핵심 기능 재확인 — 명령 + 시드 6 결과

화면 도출 전 PRD의 핵심 기능 3개를 다시 정리.

CLAUDE CODE 명령

"docs/prd.md를 읽고 핵심 기능 3개를
한 줄씩 요약해줘"

시드 6번 핵심 기능 요약

기능 1 (입력): 상품 사진 업로드
+ 핵심 키워드 5개 입력

기능 2 (핵심 처리): 카테고리별
차별화 카피 3개 후보 생성

기능 3 (출력): 스마트스토어 형식으로
복사 가능한 텍스트 출력

활동 1-3 · 산출물: 화면 2~5개 도출

화면 도출 명령 — 명령 + 시드 6 결과

핵심 2개 화면으로 압축. 우선순위 낮은 화면은 다음 사이클로.

CLAUDE CODE 명령

"docs/prd.md의 핵심 기능 3개를
사용자 흐름 순서로 화면 단위로 나눠줘.

각 화면에 어떤 기능이 매핑되는지 명시.
화면 개수는 3-5개를 넘기지 마."

시드 6번 화면 도출 결과

화면 1: 카피 입력 화면
- 기능 1 (입력): 사진 업로드 + 키워드 5
- "카피 생성" 버튼

화면 2: 카피 결과 화면
- 기능 2 (핵심 처리): 카피 3 후보 표시
- 기능 3 (출력): 카피별 복사 버튼

화면 3: 작업 이력 화면 (선택, 제외)

활동 1-4 · 산출물: 첫 화면 결정 + 한 줄 목적

첫 화면 결정 기준 — 기준 + 시드 6번 선택

페르소나 진입점 + 핵심 가설 검증 — 2개 기준.

결정 기준 (2개)

페르소나 진입점

가장 처음 만나는 화면

핵심 가설 검증

PRD 가설을 가장 빨리 시험

시드 6번 첫 화면 = 카피 입력 화면

이유:

- 셀러가 가장 처음 진입
- 핵심 가설("작성 시간 1-2시간 → 5분")
가장 빨리 검증

화면 목적 한 줄:

"1인 셀러가 신규 상품 등록 시
상품 사진과 키워드 5개를 입력해
카피 생성을 요청하는 화면"

Task 화면 단위 재구성 명령

기존 tasks.md를 화면 단위로 그룹화한 새 버전으로 교체.

"docs/prd.md와 위에서 도출한 화면 2개를 기준으로 docs/tasks.md를 다시 작성해줘.

기준:

- 화면 단위로 Task를 그룹화
- 각 Task는 1-3개 파일 수정 단위
- 화면 1을 먼저 완성하고 화면 2로 넘어가는 순서
- 의존성을 명시
- 너무 작은 Task와 너무 큰 Task 피함"

화면 1: 카피 입력 화면 — Task 4개

우선순위 1. 각 Task 1-3개 파일 수정 단위.

Task 1.1: 입력 폼 UI

- 파일: src/components/InputForm.tsx (1)
- 의존성: 없음

Task 1.2: 이미지 업로드 처리

- 파일: src/lib/upload.ts,
src/components/ImageUploader.tsx (2)
- 의존성: Task 1.1

Task 1.3: 키워드 입력 검증

- 파일: src/lib/validate.ts (1)
- 의존성: Task 1.1

Task 1.4: 입력 화면 라우팅

- 파일: src/app/page.tsx (1)
- 의존성: Task 1.1, 1.2, 1.3

화면 2: 카피 결과 화면 — Task 6개

우선순위 2. 화면 1 완성 후 진입.

Task 2.1: OpenAI API 클라이언트

- 파일: src/lib/openai-client.ts (1)

Task 2.2: 프롬프트 템플릿

- 파일: src/lib/prompts.ts (1)

Task 2.3: 카피 생성 API 라우트

- 파일: src/app/api/generate/route.ts,
src/lib/copy-generator.ts (2)

- 의존성: Task 2.1, 2.2

Task 2.4: 결과 표시 컴포넌트

- 파일: src/components/CopyOutput.tsx,
src/lib/format-copy.ts (2)

- 의존성: Task 2.3

Task 2.5: 복사 버튼

- 파일: src/components/CopyButton.tsx (1)

- 의존성: Task 2.4

Task 2.6: 결과 화면 라우팅

- 파일: src/app/result/page.tsx (1)

- 의존성: Task 2.4, 2.5

— 활동 1-6 · 산출물: 매핑 점검 완료

화면-Task 매핑 검토 (작업자 직접 점검)

4가지 기준으로 매핑 품질 점검. 화면 1 Task만으로 수직 슬라이스 첫 슬라이스 완성 여부 확인.

- 01

☒

화면 1만으로 동작 3요소 충족 가능한 구조인가
- 02

☒

Task 누락은 없는가
- 03

☒

Task가 너무 잘게 쪼개지거나 너무 크게 묶이지 않았는가
- 04

☒

의존성이 사이클을 만들지 않는가

— STEP 1 · 검증 / 성공 기준

Step 2 진입 가능한가

5개 항목 모두 통과 시 Step 2 (GitHub Issue 등록)로 진입.

- 01

☒

PRD 핵심 기능 3개에서 화면 2-3개가 도출됐는가
- 02

☒

첫 화면이 결정됐는가
- 03

☒

새 docs/tasks.md가 화면 단위로 그룹화됐는가
- 04

☒

각 Task가 1-3개 파일 수정 단위인가
- 05

☒

화면 1의 Task만으로 동작 3요소 충족 가능한 구조인가

— STEP 2

GitHub Issue 등록

docs/tasks.md → GitHub Issue + 화면 라벨링

01 2-1. Issue 등록 명령

02 2-2. 라벨 확인

GitHub MCP 설치와 인증 — Issue 등록 사전 준비

Claude Code가 GitHub API를 호출하려면 MCP 서버 설치 + Personal Access Token 인증이 필요.

1단계 · MCP 설치 명령

```
claude mcp add github ₩
  -- npx -y @modelcontextprotocol/server-github

# 또는 .claude/settings.json에 직접 등록:
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"]
    }
  }
}
```

검증: claude 재시작 후 "GitHub Issue 목록 보여줘"가 정상 응답하면 인증 완료.

2단계 · PERSONAL ACCESS TOKEN 인증

- 1 github.com → Settings → Developer settings
- 2 Personal access tokens → Generate new token (classic)
- 3 권한 선택: repo / workflow / write:packages
- 4 생성된 토큰 복사

```
# 환경변수로 등록
export GITHUB_PERSONAL_ACCESS_TOKEN=ghp_...
```

또는 .env에 저장 후 .gitignore 등록

Issue 등록 명령 — 명령 + 시드 6 결과

각 Task를 Issue로 등록하고 화면 라벨(screen-1, screen-2) 부여.

CLAUDE CODE 명령

"docs/tasks.md의 모든 Task를
GitHub Issue로 등록해줘.

각 Issue 제목: 'Task {ID}: {제목}'

본문 포함:

- 소속 화면 (화면 1 또는 화면 2)
- 수정 파일 목록
- 예상 작업 시간
- 의존성
- 작업 완료 기준

라벨:

- 화면 1 Task → screen-1 + task
- 화면 2 Task → screen-2 + task"

시드 6번 ISSUE 등록 결과 (10개)

- #1: Task 1.1: 입력 폼 UI (screen-1)
- #2: Task 1.2: 이미지 업로드 (screen-1)
- #3: Task 1.3: 키워드 검증 (screen-1)
- #4: Task 1.4: 입력 화면 라우팅 (screen-1)
- #5: Task 2.1: OpenAI 클라이언트 (screen-2)
- #6: Task 2.2: 프롬프트 템플릿 (screen-2)
- #7: Task 2.3: 카피 생성 API (screen-2)
- #8: Task 2.4: 결과 컴포넌트 (screen-2)
- #9: Task 2.5: 복사 버튼 (screen-2)
- #10: Task 2.6: 결과 라우팅 (screen-2)

라벨 확인 (4 step)

라벨로 필터링하면 화면 단위로 Task를 모아 볼 수 있다.

- 1

Issue 페이지 열기
- 2

라벨 필터에서 screen-1 선택
- 3

4개 Issue 표시 확인
- 4

screen-2 라벨로 6개 Issue 표시 확인

— STEP 2 · 검증 / 성공 기준

Step 3 진입 가능한가

3개 항목 모두 통과 시 Step 3 (Stitch로 첫 화면 시안)로 진입.

01  GitHub Issue가 10개 등록됐는가

02  각 Issue가 화면 라벨(screen-1, screen-2)을 갖는가

03  Issue 본문에 의존성과 완료 기준이 명시됐는가

— STEP 3

Stitch로 첫 화면 시안

화면 1의 UI 시안 생성 + export

- 01 3-1. Stitch 자연어 프롬프트
- 02 3-2. 시안 검토와 1회 수정
- 03 3-3. 시안 export

디자인 의도 입력 → Claude Code가 Stitch 프롬프트 생성

작업자는 의도만 짧게 입력. Stitch 프롬프트 작성 자체는 Claude Code에 위임.

작업자 → CLAUDE CODE 명령

"PRD docs/prd.md와 첫 화면 결정(1-4 산출물)을 참조해서
Stitch에 그대로 붙여 넣을 자연어 프롬프트를 만들어줘.

내 의도:

- 첫 화면 = 카피 입력 화면
- 미니멀 스타일 (Notion 톤)
- 데스크탑 기준

다음 구조로 출력:

화면 목적 한 줄 + 레이아웃 3구역 + 스타일 톤."

CLAUDE CODE 생성 결과 → STITCH에 붙여넣기

1인 스마트스토어 셀러가 신규 상품 등록 시
상품 사진과 키워드 5개를 입력해 카피 생성을
요청하는 화면.

상단에 큰 상품 사진 업로드 영역(드래그앤드롭),
중간에 키워드 입력 5칸 (가로 배열),
하단에 '카피 생성' 버튼.

Notion 같은 미니멀 스타일.

효과: 작업자가 Stitch 프롬프트 양식을 외울 필요 없음. PRD-페르소나 컨텍스트가 Claude Code를 거쳐 자연스럽게 반영.

— 활동 3-2 · 산출물: 검토 + 1회 수정

시안 검토와 1회 수정 원칙

본 단계는 동작 합격선 — 디자인 정제는 다음 단계.

- 1 생성된 시안이 페르소나에 맞는지 확인
- 2 안 맞으면 "조금 더 [형용사]게 해줘"로 1회 수정
- 3 1회 수정 후 OK 판정 (디자인 정제는 다음 단계)

시안 export — 무엇을 / 어디로

planning_project/docs/design/ 폴더에 일관 배치 (다음 Step 4 빌드 시 참조).

형식 이미지 (PNG/JPG)	무엇을 Stitch 우측 상단 "Export" → 이미지	어디에 planning_project/docs/design/screen-1.png 시각 참조용 (Claude Code 빌드 시 첨부)
형식 코드 (HTML/CSS)	무엇을 Stitch 우측 상단 "Export" → Code	어디에 planning_project/docs/design/stitch-export/ 구조 + 토큰 참조 (선택, 본 학습은 이미지로 충분)

활동 3-4 · 산출물: RULES/DESIGN.MD (다음 STEP 4 빌드 참조)

Stitch 시안 → Claude Code가 rules/design.md 작성

시안의 시각 결정(색·간격·타이포)을 rules/design.md로 정착 — 다음 빌드 명령이 자동 참조.

CLAUDE CODE 명령

"docs/design/screen-1.png (Stitch 시안)을 보고
rules/design.md를 새로 작성해줘.

다음 4축으로 추출:

- 색상 토큰 (primary / accent / bg / text)
- 간격 토큰 (padding / gap / margin 기준값)
- 타이포 (제목 / 본문 / 라벨 폰트 사이즈)
- 컴포넌트 패턴 (입력 폼 / 버튼 / 카드)

50줄 이내로 짧게.

CLAUDE.md § 5 참조 지침에도 rules/design.md 항목 추가."

RULES/DESIGN.MD (예시 결과)

디자인 가이드

색상

- primary: #1F2937
- accent: #2563EB
- bg: #FFFFFF
- text: #111827

간격

- padding 기준: 24px
- gap 기준: 16px

타이포

- 제목: 28px, font-bold
- 본문: 16px, leading-relaxed
- 라벨: 12px, uppercase

컴포넌트

- 입력 폼: 라벨 위 + 입력 박스 아래
- 버튼: bg-accent, rounded-md, py-3 px-6
- 업로드 영역: 점선 border, 드래그앤드롭

효과: Step 4 빌드 명령에서 "rules/design.md 참조"만 추가하면 시안과 일치하는 코드 자동 생성. 디자인 ↔ 코드의 일관성 자동.

— STEP 3 · 검증 / 성공 기준

Step 4 진입 가능한가

3개 항목 모두 통과 시 Step 4 (첫 Task 빌드와 PR 워크플로)로 진입.

01 ☒ 시안이 생성됐는가

02 ☒ 본인 PRD의 카피 입력 화면에 가까운가

03 ☒ export가 완료됐는가

— STEP 4

첫 Task 빌드와 PR 워크플로

Issue #1 → 위임 3원칙 → PR 생성 → 본인 리뷰 → 병합

- 01 4-1. 첫 Task 브랜치 생성
- 02 4-2. 위임 3원칙 빌드 명령
- 03 4-3. 결과 확인
- 04 4-4. 동작 3요소 부분 점검
- 05 4-5. PR 생성 명령
- 06 4-6. 본인 리뷰와 병합

활동 4-1 · 산출물: 브랜치 생성

첫 Task 브랜치 생성

브랜치 이름 형식: feat/{task-id}-{설명}. CLAUDE.md 커밋 메시지 type 그대로.

명령

```
git checkout -b feat/task-1-1-input-form
```


위임 3원칙 빌드 명령 + 명령 점검

Issue #1 명령에 3원칙이 모두 들어있는지 확인.

CLAUDE CODE 명령

"Issue #1을 진행해줘.
CLAUDE.md와 rules/architecture.md,
rules/coding-style.md를 참조해.

PRD의 카피 입력 화면을
Next.js + Tailwind로 만들어줘.
첨부한 Stitch 시안을 참조해.

- 기능은 다음 두 가지:
1. 상품 사진 업로드 영역과 키워드
입력 5칸을 렌더링
 2. '카피 생성' 버튼 클릭 시
입력값을 그대로 콘솔에 출력

실제 카피 생성 로직은 Issue #5-7에서.
이 PR은 src/components/InputForm.tsx
한 파일만 수정해."

3원칙 적용 위치 점검

범위 좁힘

"이 PR은 src/components/InputForm.tsx 한 파일만"

결과물 명시

"입력값을 콘솔에 출력"

검증 방법

"콘솔 확인으로 사전 정의"

활동 4-3 · 산출물: 동작 확인

결과 확인 — npm run dev + 브라우저 점검

로컬 dev server 띄우고 3가지 직접 점검.

명령

npm run dev

브라우저에서 확인

- 01 입력 폼이 떠 있는가
- 02 키워드 5칸과 사진 업로드 영역이 보이는가
- 03 버튼 클릭 시 콘솔에 입력값이 출력되는가

— 활동 4-4 · 산출물: 부분 충족 점검

동작 3요소 부분 점검 (첫 Task 후)

화면 1의 모든 Task(#1-#4) 완료 시 동작 3요소 완전 충족.

요소	상태
1. 페이지 뜸	충족
2. 인터랙션	충족
3. 콘솔 출력	부분

활동 4-5 · 산출물: PR 생성 명령

PR 생성 명령

Issue 자동 닫힘을 위해 본문에 Closes #1 명시.

"Issue #1 작업을 PR로 만들어줘.

PR 제목: 'feat: Issue #1 입력 폼 UI 추가'

PR 본문:

- Closes #1
- 변경 파일: src/components/InputForm.tsx
- 동작 3요소 점검 결과
- 화면 1의 첫 번째 Task

base 브랜치는 main."

— 활동 4-6 · 산출물: 첫 PR 병합

본인 리뷰와 병합 + PR 크기 원칙

GitHub 웹 흐름 6 step + PR 크기 3축 점검.

본인 리뷰와 병합 (6 STEP)

- 1PR 페이지 진입
- 2"Files changed" 탭 클릭
- 3변경 사항 검토 (CLAUDE.md/rules/ 준수 + 다른 파일 영향)
- 4"Review changes" → "Approve"
- 5"Merge pull request" 클릭
- 6브랜치 삭제

PR 크기 원칙

축	권장	피함
수정 파일 수	1-2개	5개 이상
변경 라인 수	100줄 이내	500줄 이상
한 PR의 기능	1개 Task	여러 Task 묶기

— STEP 4 · 검증 / 성공 기준

Step 5 진입 가능한가

6개 항목 모두 통과 시 Step 5 (셀프 코드 리뷰)로 진입.

01 ☒ 브랜치가 생성됐는가 (feat/task-1-1-input-form)

02 ☒ 위임 3원칙이 명령에 적용됐는가

03 ☒ 첫 화면 컴포넌트가 동작하는가

04 ☒ PR이 생성됐는가 (1개 파일 수정)

05 ☒ 본인 리뷰 후 병합됐는가

06 ☒ Issue #1이 자동 닫힘 상태인가

— STEP 5

셀프 코드 리뷰

병합 코드를 PRD/CLAUDE.md 기준으로 다시 점검

01 5-1. 셀프 리뷰 5개 점검

02 5-2. 미통과 항목 메모

셀프 리뷰 5개 점검

AI가 만들었다고 그냥 통과 X — PRD와 CLAUDE.md를 직접 대조.

01	페르소나 일치	1인 셀러
02	코딩 가이드 준수	camelCase 등
03	제약사항 위반 없음	모바일 코드 X
04	폴더 구조 원칙	rules/architecture.md 준수
05	다음 Task 로직 미리 작성 안 함	카피 생성 코드 없음

— 활동 5-2 · 산출물: DOCS/REVIEW-NOTES.MD

미통과 항목 메모

본 학습 시간 내 모두 수정 안 해도 된다. 화면 1 나머지 Task와 함께 보강.

미통과 항목 → [docs/review-notes.md](#)에 메모

예시: "Task 1.2 (이미지 업로드)에서 rules/architecture.md의 폴더 원칙 일부 위반 — 화면 1 마무리 시 보강"

본 학습 후 과제 단계로 인계 — 화면 1 나머지 Task(#2-#4) 빌드 시 같이 처리.

STEP 5 · 검증 / 성공 기준

Step N 진입 가능한가

2개 항목 모두 통과 시 Step N (정리와 비용 비교)로 진입.

01 ☒ 셀프 리뷰 5개 항목을 모두 점검했는가

02 ☒ 과제로 가져갈 항목이 docs/review-notes.md에 메모됐는가

— STEP N

정리와 비용 비교

산출물 확인 + 토큰 비교 + 회고

01 N-1. 산출물 6개 정리

02 N-2. /cost 비교

03 N-3. 회고 3개 질문

— 활동 N-1 · 산출물: 6개 정리 완료

산출물 정리 — 6개

다음 단계 활용처 명시.

1

새 docs/tasks.md (화면 단위)

다음 Task 진행 기준

2

GitHub Issue 10개

작업 추적

3

Stitch 시안 (화면 1)

빌드 참조

4

카피 입력 폼 컴포넌트

동작하는 첫 화면

5

첫 PR 병합

워크플로 완성

6

빌드 명령 메모

다음 Task 재사용

활동 N-2 · 산출물: 토큰 비교 메모

/cost 비교

Step 0 시작 토큰과 Step N 종료 토큰의 차이를 메모.

명령

/cost

시작 → 종료 토큰 차이를 본 학습 회고에 반영. 화면 1 빌드 평균 비용 베이스라인.

회고 — 좋은 답 vs 나쁜 답

구체성이 회고의 품질을 결정.

질문	좋은 답 (구체)	나쁜 답 (추상)
화면 분석 효과	"기능 단위 Task가 화면 단위로 다시 묶이면서 빌드 순서가 명확해짐"	"도움이 됐다"
위임 3원칙 효과	"한 파일만 수정하라는 범위 좁힘이 결과의 일관성 만들었음"	"좋았다"
다음 Task 패턴	"Issue #2 이미지 업로드도 같은 위임 양식으로 명령 가능"	"이어가면 된다"

— STEP N · 검증 / 완료 기준

본 학습 완료 가능한가

4개 항목 모두 통과 시 본 학습 종료 + 과제 진입.

01 ☒ 산출물 6개가 모두 손에 있는가

02 ☒ /cost 비교를 메모했는가

03 ☒ 회고 3개 질문에 모두 답했는가

04 ☒ Issue #2 시작 준비가 됐는가

과제 — 화면 1 동작 3요소 완성

화면 1이 동작 3요소를 모두 충족한 상태가 다음 단계의 출발선.

항목	내용
목표	화면 1의 모든 Task 완료와 동작 3요소 충족
산출물	새 docs/tasks.md, Issue 10개, 첫 PR 병합 + 화면 1 나머지 Task
마감	다음 학습 단계 시작 전

— TIPS 1

화면 분석 팁

기능 단위 ↔ 화면 단위의 매핑 원칙

— TIPS 1 · 화면 분석 팁

기능 단위에서 화면 단위로

화면 분석은 빌드 전 필수

PRD 핵심 기능은 기능 단위이지만 빌드는 화면 단위. 매핑이 명확하지 않은 채로 빌드 시작하면 매번 흔들림.

화면 개수 3-5개로 제한

본 학습 빌드 시간이 한정. 6개 이상이면 한 사이클 안에 완성 안 됨. 우선순위 낮은 화면은 다음 사이클로.

— TIPS 2

위임 3원칙 팁

평생 쓰는 빌드 명령 작성 습관

TIPS 2 · 위임 3원칙 팁

매 명령 전 3개 질문 습관화

모든 빌드 명령에 적용

한 번 익히면 본 학습 내내 반복 사용. 본 학습 후에도 평생 쓸 수 있는 사고 방식.

3개 질문 습관

명령 작성 전 매번: 범위가 한 화면·한 기능인가 / 결과물이 명시됐는가 / 검증 방법이 사전에 정해졌는가.

— TIPS 3

동작 정의 팁

합격선과 디자인 정제의 분리

— TIPS 3 · 동작 정의 팁

합격선만 챙기고 정제는 다음 단계로

동작 3요소 챙기기

뜸, 인터랙션, 반응. 디자인 미완성과 에러 처리 누락은 합격선과 무관.

Stitch 시안 그대로 안 옮김

시안은 참조. 실제 구현은 본인 기술 스택에 맞게 변형.

— TIPS 4

빌드 워크플로 팁

첫 화면 동작 → 패턴 복사 → 셀프 리뷰

— TIPS 4 · 빌드 워크플로 팁

첫 화면 동작이 다음 화면의 출발점

첫 화면 동작이면 절반은 끝

두 번째 화면부터는 패턴 복사. 첫 화면 빌드 명령을 메모해두면 다음 단계 출발점.

셀프 리뷰는 코드 의심하는 시간

AI가 만들었다고 그냥 통과 안 시킴. PRD와 CLAUDE.md를 직접 대조.

광범위한 명령은 의도와 어긋남

"풀스택 만들어줘" 결과는 매번 다름. 위임 3원칙으로 결과를 줍힘.

— TIPS 5

Issue·PR 팁

Task 1-3개 파일 + PR 1-2개 파일의 균형

검토 가능한 크기와 화면 단위 추적

Task 1-3개 파일, PR 1-2개 파일

너무 큰 PR은 검토 불가. 너무 작은 PR은 운영 부하만 증가. 1-2개 파일이 검토 가능한 균형.

화면 라벨로 진행 추적

screen-1, screen-2 라벨 등록. 화면 단위 진행 상황을 한눈에. PM이나 본인 추적 가능.

수직 슬라이스로 한 화면 통째

화면 1의 Task 4개 모두 완료 → 화면 2로. 병행하면 수평 슬라이스로 변질.